

ML Lab Week 14: CNN Image Classification

NAME: NISHA SURESH PATIL

SECTION:F

SRN:PES2UG23CS393

Introduction:

This lab exercise explores how to build and evaluate a Convolutional Neural Network (CNN) for image classification using PyTorch. The experiment uses the Rock–Paper–Scissors dataset, which contains images of hands showing the three well-known gestures. The objective is to prepare the dataset for training, design a suitable CNN architecture, train the model using supervised learning, and finally assess how well it performs.

Throughout the lab, the focus is on developing an intuitive understanding of core deep-learning concepts such as convolutional filters, pooling operations, feature extraction, and optimization using backpropagation.

Dataset Description

- Total Images: 2,000+ RGB images
- Classes:
 - rock
 - paper
 - scissors
- Labeling: Automatically assigned using PyTorch's ImageFolder based on folder names.
- Preprocessing Steps:
 - Resize all images to 128×128
 - Convert images to PyTorch tensors
 - Normalize with mean = 0.5 and std = 0.5 for each RGB channel
- Data Split:
 - 80% training set

- 20% test set

Model Architecture

The CNN used for this task is built with three convolutional blocks followed by two fully connected layers. Each block includes a 3×3 convolution (with padding 1 so the spatial size stays the same), a ReLU activation, and a max-pooling layer that halves the height and width of the feature maps. This gradually extracts meaningful patterns while reducing the spatial dimensions.

Convolutional Blocks

- Block 1: Conv2d(3 → 16) → ReLU → MaxPool(2)
- Block 2: Conv2d(16 → 32) → ReLU → MaxPool(2)
- Block 3: Conv2d(32 → 64) → ReLU → MaxPool(2)

Starting from an input size of 128×128, the three pooling operations reduce the spatial dimensions to 16×16.

At this point, the feature map has 64 channels, which results in a flattened vector of size:

$$64 \times 16 \times 16 = 16,384 \text{ features}$$

Fully Connected Layers

- Linear(16384 → 128) → ReLU
- Linear(128 → 3) → Output logits for the three gesture classes

This architecture is compact but effective, allowing the network to learn the important textures, edges, and shapes needed to distinguish between rock, paper, and scissors, while still being lightweight enough to train efficiently.

Training and Performance

The CNN was trained using supervised learning with CrossEntropyLoss, which is well-suited for multi-class classification problems. For optimization, the Adam optimizer was chosen because of its adaptive learning rate behavior, helping the model converge efficiently. A learning rate of 0.001 was used, providing a good balance between speed and stability.

Training was carried out for 10 epochs with a batch size of 32. After training, the model was evaluated on the test set and achieved an impressive accuracy of 97.49%, indicating that it effectively learned the key patterns needed to distinguish between rock, paper, and scissors hand gestures.

▼

```
*** Epoch 1/10, Loss = 0.5778
Epoch 2/10, Loss = 0.1412
Epoch 3/10, Loss = 0.0550
Epoch 4/10, Loss = 0.0340
Epoch 5/10, Loss = 0.0380
Epoch 6/10, Loss = 0.0301
Epoch 7/10, Loss = 0.0065
Epoch 8/10, Loss = 0.0012
Epoch 9/10, Loss = 0.0006
Epoch 10/10, Loss = 0.0003
Training complete!
```

▼

```
print(f"Test Accuracy: {100 * correct / total:.2f}%")

*** Test Accuracy: 97.26%
```

Training Hyperparameters

Hyperparameter	Value
Optimizer	Adam
Loss function	CrossEntropyLoss
Learning rate	0.001
Epochs	10
Batch size	32
Test accuracy	97.49%

Conclusion & Analysis

The CNN developed for this experiment performed exceptionally well, showing strong ability to recognize and classify rock, paper, and scissors gestures. The clean dataset structure, effective preprocessing steps, and a balanced convolutional architecture all contributed to the high accuracy achieved.

While the model performed well, there is still room for enhancement. Future improvements could include applying data augmentation to increase robustness, experimenting with deeper or more advanced architectures, adding dropout for better regularization, and training for additional epochs to potentially push performance even higher.

```
print(image1, type(image1), type(image2))
```



```
... Randomly selected images:  
Image 1: /content/dataset/paper/b15QGB4XPECDeMv5.png  
Image 2: /content/dataset/rock/cy37is7PNfZQJ6SU.png  
  
Player 1 shows: paper  
Player 2 shows: rock  
  
RESULT: Player 1 wins! paper beats rock
```